

Resumen Capa de Transporte

☰ Comentario	
☑ Completo	☐

La capa de Transporte permite a las Aplicaciones comunicarse: multiplexar y demultiplexar

Conexión entre aplicaciones.

La capa de transporte provee comunicación lógica entre aplicaciones. Comunicación lógica se refiere a que desde la perspectiva de la aplicación, es como si los hosts estuvieran directamente conectados, pero físicamente no es así porque en el medio hay routers y enlaces. El modelo de capas tiene como objeto poder entender mejor como funcionan las comunicaciones. Vamos a ir viendo que para que esto funcione es necesario que las comunicaciones ocurran entre distintas capas.

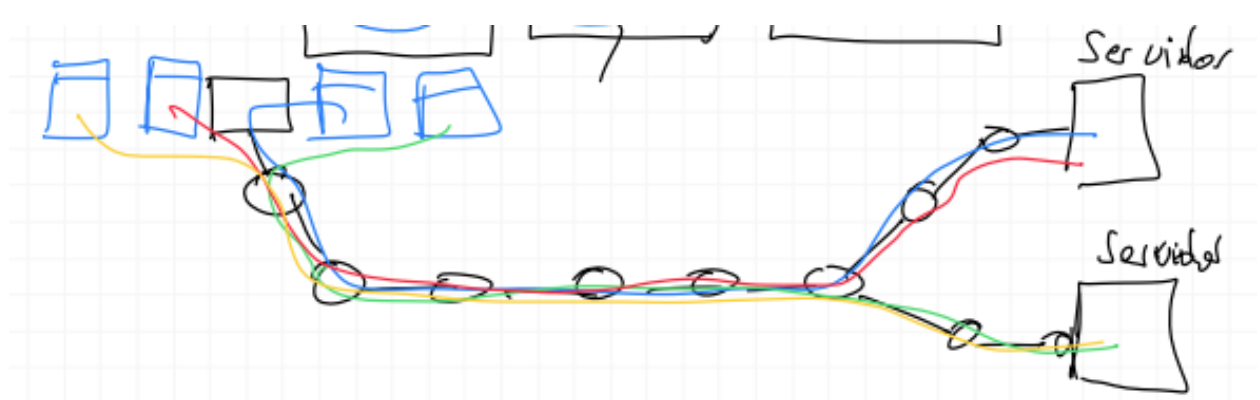
Los protocolos de capa de transporte se implementan en los hosts, pero no en los routers. Convierten los mensajes a enviar en paquetes que se conocen como segments. Este segment se le pasa a la capa de red, que lo transforma en un datagram y luego se envia al destino.

Así como la capa de transporte provee comunicación lógica entre aplicaciones (procesos), la capa de red provee comunicación lógica entre hosts. El protocolo de transporte mueve la información hasta el borde de la red (o sea, la capa de red) y viceversa pero no tiene nada que ver en cómo esa información viaja dentro de la red. El **servicio fundamental** de la Capa de Transporte consiste en multiplexar y demultiplexar.

Multiplexar y demultiplexar

La principal responsabilidad de los protocolos UDP y TCP es la de extender el servicio de entrega entre dos hosts a un servicio de entrega entre dos procesos corriendo en

dichos hosts. Esta extensión se llama transport-layer multiplexing and demultiplexing.



La capa de transporte no le entrega los datos directamente al proceso, sino que al socket. Cada socket tiene un id. Para saber a qué socket debe entregarle, cada segmento tiene un set de campos donde se especifica el id del socket. Esta acción de entregar datos de la capa de transporte al socket es demultiplexing. La acción de juntar pedazos de datos de diferentes sockets, encapsularlos con un header para crear segmentos y pasarselos a la capa de red, es multiplexing.

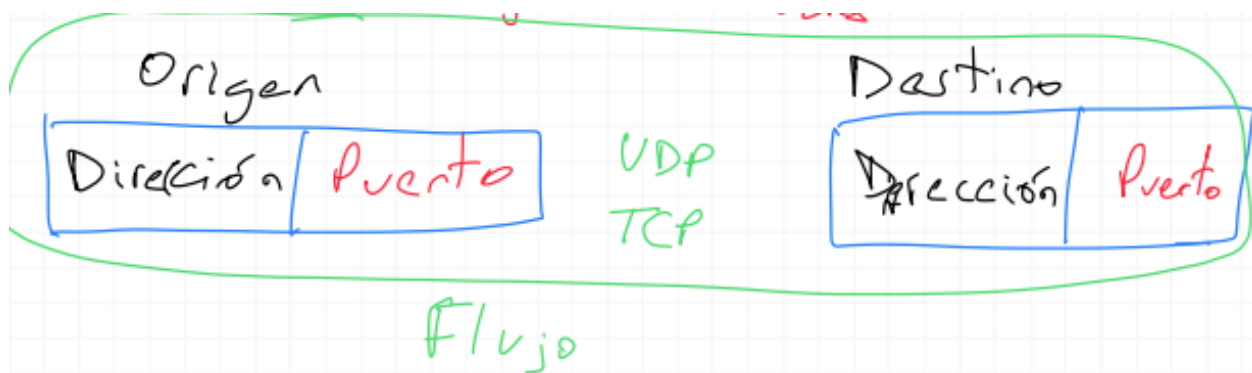
Multiplexar: Juntar en un mismo medio varias comunicaciones, consultar muchos datos a travez de un mismo canal, hacer lo contrario seria demultiplexear.

En el header deben estar seteados los campos: source port number field y destination port number field. Así al hacer el demultiplexing, la capa de transporte se fija en el destination port a qué socket debe mandar el paquete. El source port sirve por si se quiere enviar algo de retorno.

Transporte en internet: Best effort (intenta lo mejor que puede)

- **Best effort:** es una propiedad heredada de la capa de Red. Es decir hacer lo mejor posible, ademas como vemos en el modelo de capas, los mensajes en las capas intermedias nunca sobrepasan la capa de Red. Esto es porque lo que se esperaba era hacer una red cuyos dispositivos intermedios sean económicos (rústicos por la epoca), esto permitió que internet creciera de manera enorme.
 - **Facilita el diseño** de la red: equipos + simples, protocolos - complejos.

- Hizo **crecer** a Internet. Hasta dominar la tecnología de redes de comunicaciones.
- Proporciona **multiplexado** y **verificación de errores mínima** (Checksum).
- Puede proveer:
 - **Confiabilidad** de las comunicaciones:
 - Establecer la comunicación full duplex (full duplex: debe establecerse la comunicación en ambos sentidos; half duplex: primero habla uno y luego habla el otro; simplex: solo uno puede hablar y el otro debe escuchar)
 - Que llegue todo, pudiendo asegurar integridad y orden. Lo que no llega se debe reenviar.
 - **Control de flujo:**
 - Taza de transmisión: es un acuerdo entre los dos extremos para establecer una velocidad en la que ambos puedan recibir/transmitir información.
 - Ancho de banda efectivo. Control de contagio.
 - **Seguridad:**
 - Verificar los endpoints.
 - Privacidad: resguardar información.
 - Integridad de los datos: evitar que los datos sean corrompidos.

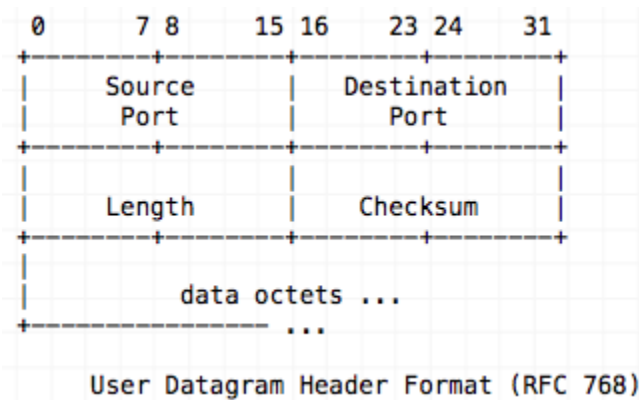


Lo que se hace para mandar un paquete es poner una dirección IP del origen y del destino, y se agrega el puerto que es lo que nos permite multiplexar tanto en el origen y en el destino. Los puertos en el caso de los servidores están relacionados con las

aplicaciones, el puerto destino es el mismo para una misma aplicación, pero el puerto origen debe ser distinto (sino no se puede diferenciar entre las distintas tabs).

Un flujo de datos es el conjunto de: Origen {dirección IP, puerto} - Destino {dirección IP, puerto} - Protocolo {TCP o UDP}. El flujo es distinto si un paquete sale del cliente al servidor que si va del servidor al cliente.

Capa de transporte UDP(User Datagram Protocol)



Recordar que UDP es connectionless (no hace handshaking antes de enviar un segmento). El servicio que ofrece es mínimo y no le agrega nada al protocolo IP. No permite ofrecer comunicaciones fiables, es muy simple, hace lo mínimo e indispensable. DNS es un ejemplo de una aplicación que suele usar UDP. DNS envía una query y si no obtiene respuesta puede intentar enviarla de nuevo, intentar enviarla a otro servidor o informar a la aplicación que no pudo obtener respuesta.

El checksum se aplica sobre todo el mensaje, si hay un bit que cambia lo detecta, pero si hay dos bits en la misma posición no los detecta ⇒ muy limitado. El checksum da una pequeña información respecto si hay un error o no.

Por qué un developer elegiría UDP antes que TCP?

- **Más control sobre qué datos se envían y cuándo.** UDP apenas un proceso envía información, la empaqueta y la manda; TCP tiene el sistema de control de congestión que lo deja esperando si están muy cargados los enlaces. Además, como TCP garantiza que todos los paquetes lleguen y lleguen ordenados, intenta reenviar la información hasta confirmar que llegó bien, independientemente del tiempo que eso demore. Para aplicaciones de tiempo real, eso no está bueno. UDP es que nos permite mandar paquetes cuando querramos ⇒ no hay control de flujo.

- **No establece ninguna conexión.** Establecer una conexión introduce delays.
- **No mantiene estados de conexión.** Esto permite que se puedan mantener más clientes activos en una aplicación que corre con UDP.
- **Headers con bajo overhead.** TCP tiene 20 bytes header overhead, UDP tiene 8 bytes.

Que aplicaciones usarían UDP? Las aplicaciones que no requieren de servicio confiable y requieran de baja latencia como DNS, juegos multijugadores, etc.

Cabe mencionar que es posible hacer que una aplicación que se comunica con UDP transporte datos de manera confiable. Para eso debe construirse esa reliability desde la misma aplicación.

Principios de las comunicaciones confiables

- Conectadas vs. No-conectadas: es necesario que sea conectada para que sea confiable ya que con esta conexión sabemos que puede ser ordenada y tener integridad.
- Asegurar la **entrega**.
- Asegurar el **orden**: los datos estan en el mismo orden que me los mandaron.
- Asegurar la **integridad**: no contiene errores.
- **Desempeño**: Cantidad de información que puedo mandar en función de la disponibilidad de la Red. Cómo hago para enviar esas páginas a la mayor velocidad posible para que lleguen cuanto antes.
- **Control de flujo**. Que el receptor pueda recibir lo que yo quiero mandar y que yo este dispuesto a mandar. Alguien controla para que yo no me exceda y no bloquee el canal.
- **Compartir** el canal **equitativamente**.

Tipos de flujo

- Stop & Wait (desempeño). _Ej. DNS._
- Continuo (desempeño).
 - Go-Back-N.

- Selective Repeat.

Stop and Wait:

El cliente envía el paquete numero 1, el servidor me contesta con un ACK1, recién cuando el cliente recibe el ACK puede enviarle el paquete 2.

Hay un problema el tiempo en el que tardo en mandar un segmento de datos, la tasa total de transmisión es muy pequeña y va a depender de la distancia entre el cliente y el servidor. La tasa de transmisión de datos depende de la cantidad de bytes dividido un tiempo, como el tiempo total es grande, entonces la tasa es baja $\Rightarrow$ problema de desempeño del stop and wait.

La performance es: $P = T_{seria} / (T_{seria} + T_{prop})$, T_{seria} : es el tiempo de serializacion, cuando meto bits en un enlace, el enlace trabaja a una cierta velocidad, no puedo meter bits mas rapido o mas lento que el enlace, tiene que ser a exactamente esa velocidad; T_{prop} : es el tiempo de propagación total.

En otros terminos: $P = \frac{\frac{long[b]}{v[b/s]}}{\frac{long}{v} + RTT}$, long: es la longitud del mensaje; v: es la velocidad del enlace; RTT: es el round trip time.

El stop and Wait no funciona porque el RTT es mucho mayor que el tiempo de serializacion. Podemos tener el problema de que se caiga un paquete $\Rightarrow$ hay un **timer** y cuando se vence el time out y vuelve a mandar el paquete.

La manera de trabajar con una buena performace seria que el cliente pueda mandar muchos paquetes al mismo tiempo, a medida que el servidor recibe va contestando con los ACK. El problema cuando se pierde un paquete, se resuelve de dos maneras:

Go Back N

Por un lado tenemos la ventana de transmision, lo que hace es decir que puedo transmitir N segmentos, esto esta calculado para que apenas termine de enviar los paquetes me comiencen a llegar los ACKs. Ni bien me llega un ACK yo puedo desplazar la ventana en una posición y por ende puedo transmitir un paquete mas. De esta manera nunca paro de transmitir paquetes. El tamaño de la ventana esta ligado al RTT. Si siempre tardara el mismo tiempo en ir y venir yo podría calcular el tamaño exacto de la ventana para no detener nunca(caso ideal que no existe).

Cuando se pierde un paquete de datos enviado por el cliente, el servidor envía un ACK del último paquete recibido en orden hasta que le envíen el paquete que le falta. El cliente comienza a recibir **ACKs duplicados**, esto es un indicio de que se están perdiendo paquetes. Por lo que se debe reenviar el paquete perdido.

Como se hace el control de flujo? El control de flujo se acuerda entre el transmisor y el receptor, porque el receptor le dice que puede recibir hasta N segmentos, entonces el transmisor sabe que no puede abrir la ventana en más de N $\Rightarrow$ El tamaño óptimo de la ventana depende del RTT. Todas las aplicaciones interactivas mandan muchos pequeños paquetes y tienen un RTT bajo.

Aclarar que cuando se pierde un paquete se renvia toda la ventana en SR solo renvia un paquete?

Selective Repeat:

Cuando se pierde un paquete de datos enviado por el cliente, el servidor envía un ACK del último paquete recibido (por más de que este fuera de orden) y un **NAK** del paquete faltante, avisa que recibió el último pero no el anterior. Por lo que se debe reenviar el paquete perdido.

En el caso de Go Back N como en Selective Repeat si se pierde un ACK, esto no genera la retransmisión del paquete ya que por ejemplo: le llega ACK2, ACK3, ACK5 ya sabe que el paquete 4 le llegó por más de que no le haya llegado el ACK4.

Resumen de las variables y los mecanismos del flujo confiable en datos:

- **Checksum** o **CRC**: verificación de integridad.
- **Temporizador** (Timer): para detectar paquetes perdidos (puede haber duplicados).
- **Número de secuencia**: para mantener un flujo de paquetes y detectar los perdidos.
- **Acuse de Recibo (ACK)**: para avisar qué paquetes se han recibido.
- **Acuse Negativo de Recibo (NACK)**: para avisar que el paquete llegó corrupto.
- **Ventana deslizante**: para implementar un flujo de datos con alto desempeño.

TCP (Transport Control Protocol)

Características de TCP

- Los pedazos de información que se envían se denomina segmentos, deben estar numerados.
- Conexiones punto a punto.
- Actúa sobre ambos extremos.

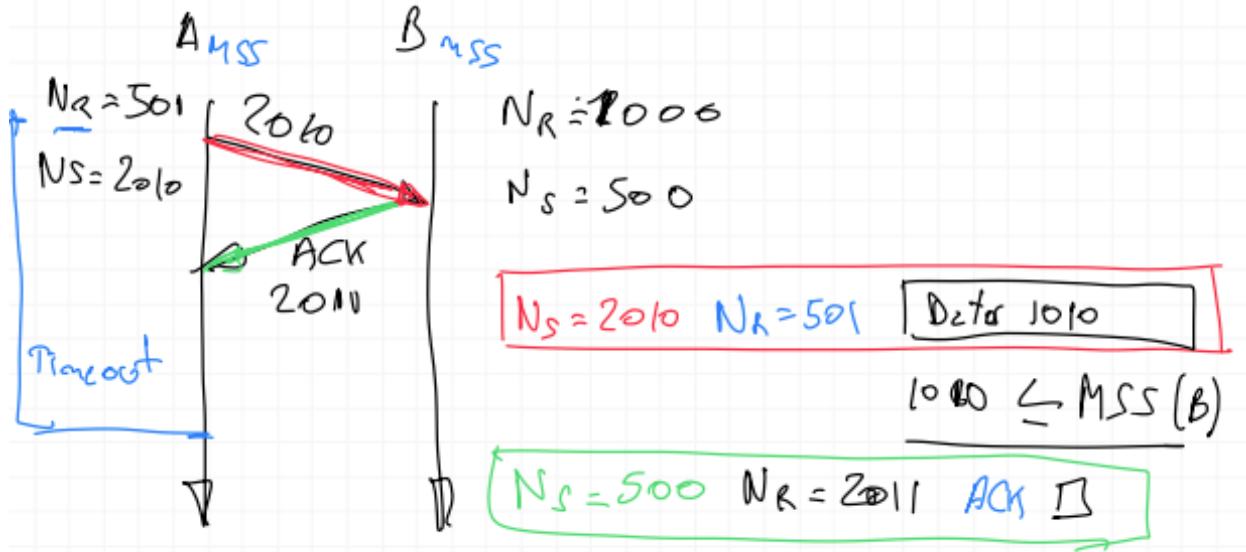
Segmentos

| ¿Cómo se numeran?

- Tamaño **variable**.
- Unidad de medida: **byte**.
- Se **numera la unidad de medida**. En este caso el byte.
- Se **inicializa** la secuencia con un **número random**.

TCP tiene una forma de enumerar particular: los segmentos de información no tiene que ser todos del mismo tamaño, sino estaríamos diferenciando entre aplicaciones. Cada una de las aplicaciones manda el tamaño que necesita. TCP encontro la manera de enumerar: enumera los bytes que va enviando. Hay un tamaño maximo de bytes(segmento) que se pueden enviar.

El tamaño maximo del segmento se define en base al siguiente parametro: tamaño del buffer de recepción. Por lo que se define el parametro MSS(Maximum segment size) que es el tamaño maximo de seg. Cada lado va a tener su propio MSS, que no necesariamente van a ser iguales ya que estan referidos a la reserva de buffer que tenga cada uno. Esto lo que me dice es que por cada paquete que envio, voy a tener un tamaño maximo.



Por esto, A y B deben estar coordinados, cuando A le envia algo a B, B debe saber que numero de paquete le esta enviando. Pero no se trabaja por numero de paquete, sino que la unidad de medida utilizada en TCP es el byte. Cada extremo tiene un numero de recepci3n(N_R), que es la cantidad de bytes que recibió hasta el momento y un numero de secuencia(N_S) que es la cantidad de bytes que envio hasta el momento. Por ejemplo tenemos el siguiente caso:

A le envia un paquete a B con 2010, es decir que hasta ahora mando hasta el byte 2010 $\Rightarrow$ B: $N_R = 1000$ y $N_S = 500 \Rightarrow$ B recibe el paquete y manda una ACK 2011, es decir que el proximo byte que desea recibir va a ser el 2011. Del lado A tenemos A: $N_R = 501$ y $N_S = 2010$.

El paquete 2010 enviado por A debe tener la siguiente informaci3n:

- $N_S = 2010(A)$
- $N_R = 501(A)$
- Dato 1010(2010-1000): $1010 \leq MSS(B)$

El paquete ACK 2011 enviado por B debe tener la siguiente informaci3n:

- $N_S = 500(B)$
- $N_R = 2011(B)$
- flag: ACK

- Datos: vacío

La idea que se utiliza en TCP es que cada vez que me mandan información, devuelvo un ACK. Esto se hace para asegurarse que la información llegó bien, principalmente en aplicaciones interactivas. Sin embargo, hay algunos casos en donde el lado B, si se trata de un archivo muy grande, espera a ver si le llega más información para enviar el ACK, esto se llama ACK delayed, son ACKs demorados. Pero no se puede demorar mucho, el tiempo que debe esperar es el timeout.

Distintas situaciones:

El lado A envía un paquete y se lea y lanza el timeout. Si por alguna razón el ACK viene después del timeout, A piensa que lo perdió entonces lo vuelve a enviar. En el momento en que A recibe el ACK envía el siguiente paquete.

Razones por las cuales no llega el ACK: puede pasar que se pierda el paquete; que el paquete llegue y se pierda el ACK $\Rightarrow$ diferentes estrategias. En GBN, el lado que está transmitiendo comienza a retransmitir toda la info y en SR solamente se retransmite aquello por lo cual no se recibió el ACK. Pero con esta idea de los ACKs acumulados no podemos usar un SR tan fácilmente. Que tal si empiezo a retransmitir el segmento que se perdió (o el segmento perdido o el ACK perdido).

Un router descartaría paquetes porque está muy saturado y los dropea ya que no le entran en el buffer. Pero se dieron cuenta que si esperaban a que el buffer esté completamente lleno para descartar paquetes, se iban a descartar muchos paquetes. Por lo que se generó un sistema en el que cuando el buffer llegaba a un porcentaje elevado de utilización, el router elimina algunos paquetes.

A un router lo atraviesan muchos flujos al mismo tiempo, entonces aun en el caso de que el router reaccionara cuando el buffer está completo, los paquetes que se descartan van a ser de muchos flujos distintos. Por lo que el router le va a comunicar a los distintos flujos que le llegan que hay un inconveniente en la Red. Los flujos frente a esto disminuyen su tasa de envío para evitar seguir congestionando al router. Esto hace que se comparta la capacidad de los enlaces y del router de enrutar los paquetes. Eso hace que varios flujos TCP puedan compartir la Red y sus recursos. El router desconoce a los flujos que pasan por él, por lo que en esta concepción de TCP, eliminar los paquetes es la manera de avisarle a los extremos de los flujos la congestión que hay. Estos conceptos llevan a un diseño de una red eficiente y simple

con algunas limitaciones. Con la ventaja de que el trafico se puede adecuar a los recursos que estan disponibles.

Otra situacion: el que transmite, transmite muchos paquetes y cada vez que transmite un paquete enciende un timer distinto. Cuando llega la respuesta desactiva el timer. El problema esta cuando se pierde un paquete, ya sea el paquete o la respuesta. El que recibe no sabe que es lo que se perdio. En este caso en particular, a medida que recibimos respuestas, si se pierde el ACK3 y recibe el ACK4 $\Rightarrow$ sabemos recibio todo lo anterior correctamente y que se perdio el ACK. En el caso de tener ACKs duplicados sabemos que se perdio el paquete, ya que si se pierde el paquete 2, voy a estar recibiendo ACKs 1 ya que fue el ultimo en recibirse correctamente(en orden) hasta que me envíe el paquete 2.

La red no asegura nada, ni que los paquetes lleguen al destino, ni que lleguen en orden, ni que lleguen por el mismo camino, ni nada. Entonces si recibo un ACK1 y luego un ACK duplicado, esto lo que me dice es que estan llegando paquetes pero en desorden, falta el paquete 2. Un ACK duplicado no es garantia de que el paquete se haya perdido, ya que este puede estar demorado en la Red. No hay una garantia pero a medida que me llegan mas ACKs duplicados hay mas chances de que el paquete se haya perdido.

Para calcular el timeout, lo que uno intenta hacer es estimar el RTT, que es el tiempo minimo que tarda la informacion en ir y volver. El paquete/respuesta puede viajar a distinta velocidad segun la cantidad de trafico que haya en la red, no es cte. Por lo que cuando queremos calcular el RTT que valor se debe usar?

TCP usa el mecanismo de timeout/retransmisión para los segmentos perdidos. Recordar que el servicio IP no es confiable: no puede garantizar que los datos lleguen, ni que lleguen en orden, ni que no se corrompan en el camino. Pero TCP sí debe garantizar eso, para ello tiene que construir un servicio de transferencia confiable por sobre el servicio IP. Una recomendación es que se use un solo timer de retransmisión (porque tener uno por cada segmento no reconocido, implica mucho overhead)

Cada vez que TCP retransmite un segmento, duplica el intervalo de timeout. Esto provee una especie de control de congestión limitado. Uno de los problemas de las retransmisiones triggereadas por un timeout, es que el periodo de timeout puede ser relativamente largo. Existe una manera que tiene TCP para saber que un paquete fue perdido antes de que termine el timeout $\rightarrow$ ACK's duplicados. Un ACK duplicado es un

ACK de un paquete del cual el sender ya recibió reconocimiento. Cuando el receiver recibe un segmento con un número de secuencia mayor al esperado, detecta que falta un segmento (por perdida o por desorden); ante esto, reenvía una señal ACK por el último byte de datos recibido correctamente. Si el sender recibe 3 ACKs del mismo segmento, interpreta que el segmento siguiente al que fue ACKd tres veces, se perdió y hace un fast retransmit, o sea retransmite antes de que termine el timeout.

Timeout

¿Cómo se calcula?

RTT 1: Tenemos una primer estimación con los paquetes del handshake. **Variaciones enormes** con una sólo medida.

El tiempo minimo que hay que esperar se denomina RTT y es el tiempo que tarda la informacion en ir y volver. Como depende de la cant de trafico que haya en la red, el tiempo de llegada puede ser muy distinto $\Rightarrow$ el RTT varia. Para calcular el timeout se utiliza un calculo estadistico:

Calculo

- $T_{(i)} = \alpha * RTT_{(i)} + (1 - \alpha) * T_{(i-1)}$
 - Normalmente, $\alpha = 1/8$.
 - Esto hace un promedio ponderado exponencialmente, las mediciones mas antiguas siguen pesando en la estimacion actual.
- $DT_{(i)} = |RTT_{(i)} - T_{(i)}| * \beta + (1 - \beta) * DT_{(i-1)}$
 - Normalmente, $\beta = 1/4$.
 - $DT_{(i)}$: desvio de $T_{(i)}$
- $RTO_{(i)} = T_{(i)} + 4 * DT_{(i)}$
 - $RTO_{(i)}$: Retransmission Timeout para el paquete i.
- Estamos suponiendo un modelo de distribucion(varianza acotada).
 - **Modelo Poissoniano.** En la realidad **el trafico no sigue una distribucion de Poisson.**

En general, el modoo poissonneano no sirve. La idea es que unicamente tengamos timeout ya se cuando se pierde el paquete que va o el que viene. Tambien sabemos que si lo que se pierde son los ACKs tenemos la posibilidad de recibir el ACK siguiente sin necesidad de retransmitir la informacion.

Transmision confiable

¿Cómo se logra?

Si el desvio es pequeno, entonces voy a obtener un timeout muy chico $\Rightarrow$ si el ACK no me llega casi al instante, supongo que tengo que retransmitir. Si la variacion entre los RTT es grande entonces el timeout es grande $\Rightarrow$ ACKs repetidos. Al momento de que me llega el 3er ACK repetido, entonces supongo que se perdio un paquete. La idea es que con los ACKs repetidos pueda detectar antes del timeout que hay un paquete perdido y no tener que esperar al timeout. Si llega al timeout y no me llega ningun ACK repetido $\Rightarrow$ nunca llego ningun paquete.

Algoritmo RED: Random Early Discard la idea es que antes que se llene el buffer se comiencen a descartar paquetes al azar y eso hace que algunos flujos empiecen a adaptarse y evita que se pierdan un monton de paquetes al llenarse el buffer.

Puede ocurrir que haya un orden distinto de los paquetes $\Rightarrow$ algun ACKs repetidos pueden suceder porque ocurren los reordenamientos de los paquetes. Como nos conviene operar? Si yo recibo 3ACKs repetidos es muy probable que haya perdido el paquete por lo que se reenvia este paquete. Uno puede enviar una cantidad de paquetes segun la ventana, en la medida que yo recibo un ACK se puede desplazar la ventana en uno. Porque le ACK me dice que sabemos que llego correctamente el paquete. El problema es cual es el tamaño optimo de la ventana? Depende de lo que puede recibir el receptor y del RTT(que no es cte). Si el RTT fuera cte, la ventana optima tendria que ser una ventana que me permitiera seguir enviando informacion hasta el primer ACK recibido.

Hay una relacion de la ventana de transmision(W) con el RTT, la velocidad (throuput), del MSS y del buffer de recepcion($rwin$) $W = f(RTT, throughput, MSS, rwin)$

Analisis de escenarios

- Timeout.

- **Fast Retransmit y Fast-Recovery.**
- **Control de flujo.** Espaciar el envío.
 - **Ventana de recepción:** Tengo que decirle al otro extremo cual es mi **ventana actual de recepción**, para que el otro extremo sepa lo que me puede enviar.
 - **MSS (Maximum Segment Size).** Puede variar en cada lado. `rwin = numero * MSS`.

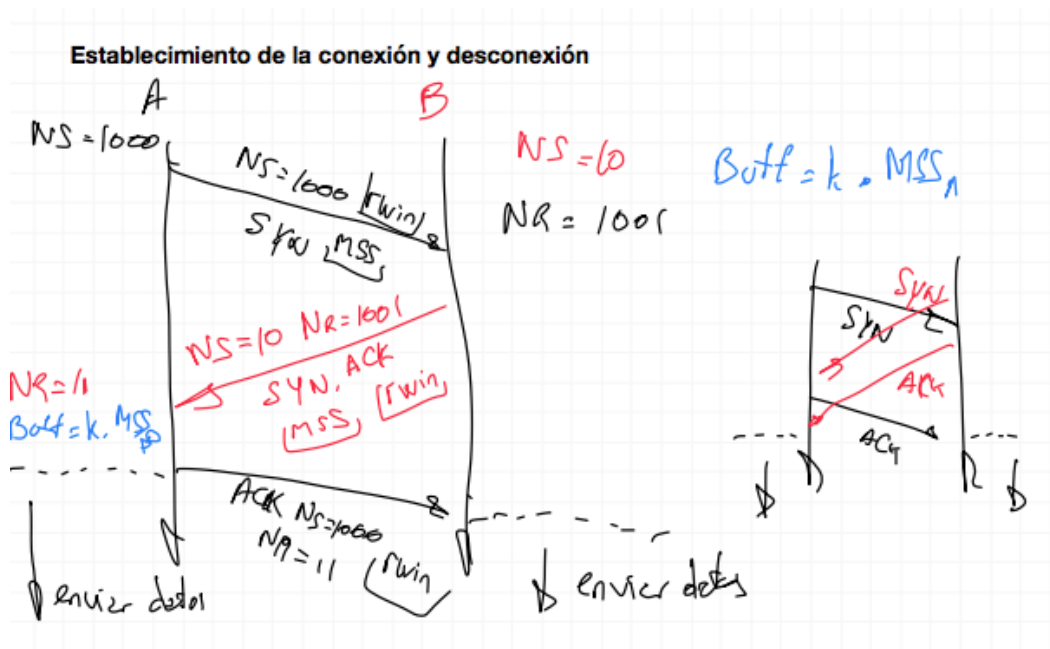
Establecimiento de la conexión y desconexión

Se dice que el protocolo TCP es connection-oriented porque antes de que un proceso pueda comenzar a enviar datos a otro, debe establecerse la comunicación entre ambos (handshaking). Esta primera comunicación sirve para inicializar variables de estado asociadas a la conexión.

Una conexión TCP provee un servicio full-duplex: si hay una conexión TCP entre un proceso A en un host y un proceso B en otro host, entonces los datos de la capa de aplicación pueden viajar de A a B al mismo tiempo que datos viajan de B a A.

Una conexión TCP es siempre point-to-point: entre un único sender y un único receiver.

La conexión se establece primero con un three-way handshake. Establecida la conexión, el cliente envía información a través del socket. Pasado el socket, los datos quedan en manos de TCP del lado del cliente. TCP dirige los datos al buffer de envío. Cada tanto, TCP va a tomar chunks de datos del buffer y se los va a pasar a la capa de red. La cantidad máxima de datos que puede tomar y enviar está limitada por el tamaño máximo del segmento (MSS-maximum segment size). TCP empareja cada chunk de data con un header, formando segmentos. En resumen, una conexión TCP consiste de buffers, variables y sockets en ambos hosts.



Para establecer la conexión, entre el extremo A y B se debe:

Three Way Handshake:

1. A(cliente) le manda a B(servidor) el primer paquete: flag **SYN** encendido. Además en ese paquete:
 - A le avisa a B cuál es su NS (número de secuencia).
 - A le avisa a B cuál es su MSS. Cada extremo comunica su MSS. Pueden ser distintos. Cuánto espero recibir por segmento.
2. El paquete llega al servidor.
 - a. El server setea el NR (contador de recepción).
 - b. Inicia su NS (secuencia propio).
 - c. Reserva un buffer de recepción igual a: $Buff = k \cdot MSS_A$.
3. El server le contesta al cliente. Le envía un flag de SYN, flag de ACK, su MSS, su NS, y su NR y su parametro rwin (ventana de recepción).
4. El cliente recibe el paquete.
 - a. Setea el NR con la información que le viene (NS del servidor + 1).
 - b. Reserva un buffer: $Buff = k \cdot MSS_B$ (del servidor).

5. El cliente envía un ACK. Con su NS, el NR y su rwin.
6. A partir de este momento, pueden enviar datos.

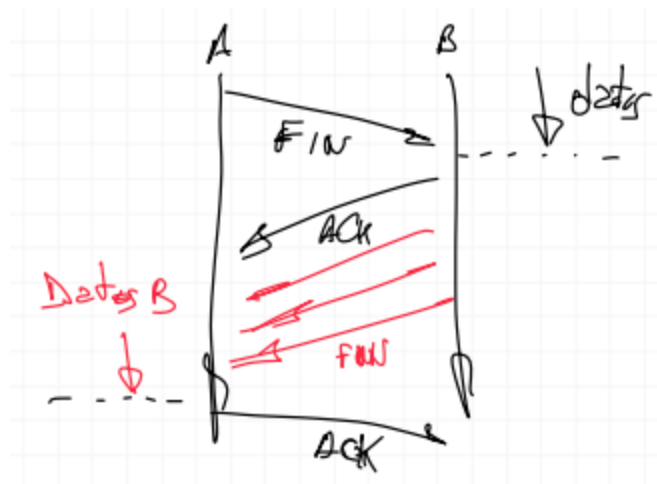
Cuestiones

- En el encabezado sólo debe ir lo fundamental. El MSS no es necesario.
- Se puede establecer la conexión si ambos se envían un sync. TCP no pone restricciones sobre quien inicia la comunicación, por lo que se puede dar una apertura simultanea.
- **Simetría:** Ambos pueden ser cliente y servidor al mismo tiempo. Una vez que se abrió la conexión, cualquiera puede transmitir o recibir.

Cerrar la conexión

1. A le envía un paquete con el flag **FIN** a B.
2. El otro extremo lo recibe y envía un ACK.
3. Desde que B recibe **FIN**, A ya no puede enviar pero puede seguir escuchando y B puede seguir enviando.
4. Cuando B envía un **FIN**, A envía un ACK y se cierra la conexión.

Obs.: se puede enviar un **RESET** para cerrar ambos extremos.



Control de congestión

Regula la cantidad de datos que se estan mandando.

- **Ventana deslizante.** Cantidad de bytes que podemos enviar.

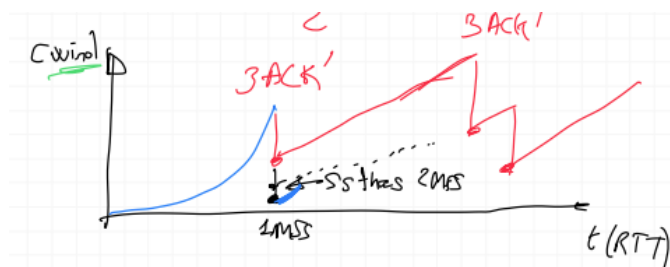
Definiciones

- $W = \text{LastByteSend} - \text{LastByteAcked}$.
- $\text{rwind} :=$ ventana de recepción.
- $\text{cwind} :=$ ventana de congestión.
- La ventana que enviamos es: $W = \min(\text{rwind}, \text{cwind})$.

Init:

- $\text{cwind} = 1 \text{ MSS}$ [bytes].
- $\text{ssthres} = 65535$ (slow start threshold)[bytes].
- Si $\text{cwind} < \text{ssthres} \Rightarrow$ **Slow start**
- Si $\text{cwind} \geq \text{ssthres} \Rightarrow$ **Congestion avoidance**

Arrancamos con una ventana chica, 1MSS, transmitimos un segmento, arrancamos en slow start. Cuando recibimos el ACK lo que dice al algoritmo de Slow Start es que se duplica la ventana $\Rightarrow$ se envían 2 segmentos. Cuando recibo el ACK de los dos $\Rightarrow$ puedo mandar 4 segmentos. Es decir que por cada ACK que recibo puedo incrementar mi ventana en 1MSS. Puede suceder que perdamos un paquete $\Rightarrow$ si hay 3 ACKs repetidos entonces podemos decir que se perdio un paquete $\rightarrow$ actualizamos la ventana de congestion $\text{cwind}(i) = \text{cwind}(i-1)/2$ y $\text{ssthres} = \text{cwind}(i)$. En el grafico la linea azul es con slow start, luego cae por los 3 ACKs repetidos. Cuando estamos en Congestion Avoidance lo que hacemos es en vez de incrementar en 1MSS lo aumentamos de a 1/1MSS. Cuando hay 3ACKs repetidos vuelve a caer el cwind y luego vuelve a crecer linealmente con Congestion Avoidance.

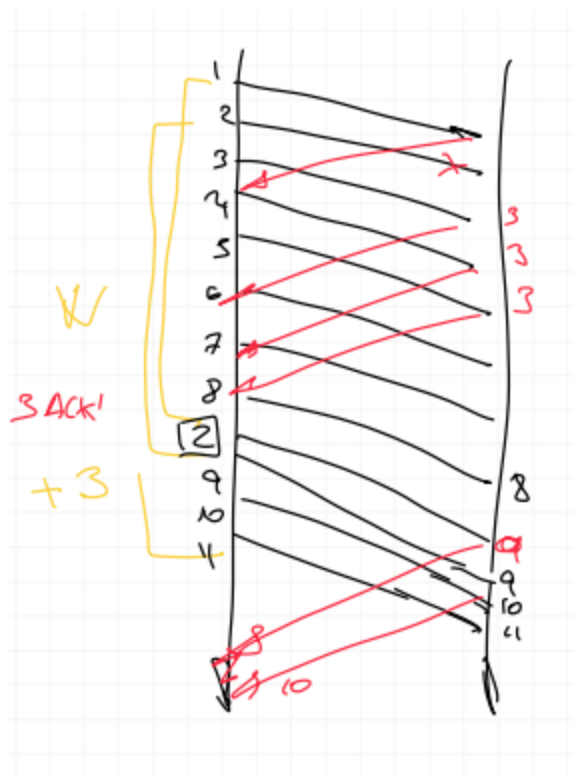


La ventana con la que se transmite es una ventana expresada por el minimo de ambas ventanas(congestion y recepción del otro extremo). En este grafico estoy suponiendo que el otro extremo tiene una ventana de recepción enorme, mucho mas grande de lo que puedo tener.

Situacion particular: Timeout

Cuando hay un timeout, el cwind cae a 1MSS y el ssthres cae a 2MSS esto hace que no podamos arrancar en slow start sino en congestion avoidance[parte en negro del grafico de arriba]. La ventana se reduce tanto porque esta todo tan congestionado que hay que tener una situación extrema. Estamos en slow start por 1 RTT, se ve en la curva marcada en azul.

Fast recovery/Fast retransmit

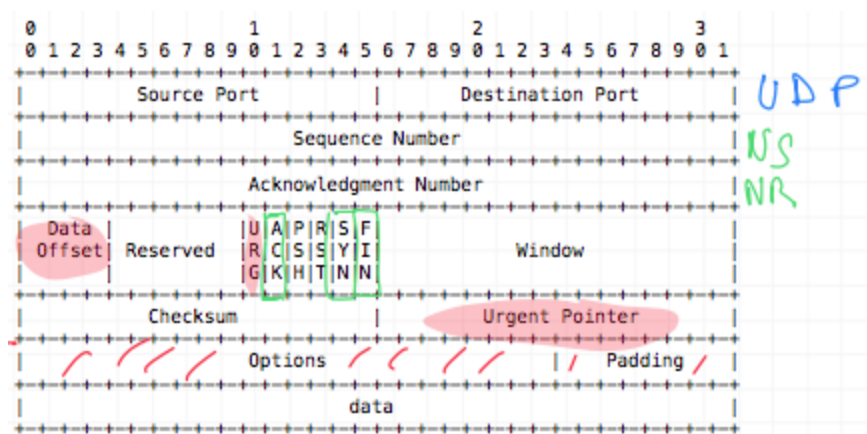


Cuando me llega el 3 ACK repetido, agranda la ventana en 3MSS, ya que ya se que se perdio el segmento 2 pero que llegaron bien los 3 segmentos siguientes. Por lo que para no perder el ritmo de mandar paquetes se incrementa la ventana en 3 para seguir mandando los paquetes 9,10,11.

Encabezado de TCP

La primera línea es igual a la de UDP. La línea 4 tiene los distintos flags, unos bits reservados, window es la ventana del que transmite, le esta especificando a quien recibe el mensaje cual es la ventana de recepcion. Cada vez que se manda un dato o un ACK se manda la ventana de recepcion.

El flag URG, indica que un segmento envia datos que deben comunicarse de manera urgente y se utiliza el Data Offset y Urgent Pointer para indicar que parte de la data es la urgente y cual no. Los bits de options se utilizan durante el three way handshake(se usa cuando el sender y receiver negocian el tamaño máximo del segmento) y el data offset se utiliza para saber en donde arrancan los datos de option.



Los bits RST, SYN y FIN son para el setup y teardown de la conexión.

Los bits CWR y ECE se usan para notificar congestión.

El bit PSH indica al receiver que debe pasar los datos inmediatamente a la capa superior.

El bit URG indica que hay datos marcados como urgentes por la capa superior del sender.

El bit ACK indica que el valor cargado en el campo de acknowledgment es válido.

Cuando un host envía un segmento con el bit RST en 1, le está diciendo que no tiene un socket para ese segmento y que no lo reenvíe.

La **ventana de congestion** esta relacionada con las posibilidades de la Red, cuanto dispone la Red para que yo utilice. Mientras que

la **ventana de recepcion** esta relacionada al control de flujo. El **control de flujo** lo hace el receptor(A o B), mientras que el **control de congestion** lo hacen los transmisores.

Control de flujo

Regula cuanto le manda el receptor al transmisor, el receptor le puede mandar una ventana(rwnd) de tamaño cero $\Rightarrow$ el transmisor debe dejar de mandar datos por un cierto tiempo. Cuando el transmisor vuelve a mandar un mensaje lo manda con el flag PUSH, lo cual le indica al receptor que debe responder inmediatamente. Si el receptor esta disponible le envia su nueva rwnd.

- TCP **intenta** ser parejo. Pero no siempre lo es.
- Un enlace congestionado **afecta a todos** los flujos que pasan por este enlace.

LIBRO:

Existe también una versión modificada de TCP donde provee selective acknowledgment. Esto significa que el receiver puede reconocer segmentos noordenados de manera selectiva. Combinado con selective retransmission, hace que TCP se asemeje más al tipo Selective-Repeat.

Se dice que el mecanismo de errores de TCP es un hibrido de GBN y SR.

TCP provee un servicio de control de flujo para que el sender no colapse el buffer del receiver. Esto se logra haciendo que el sender mantenga una variable llamada receive window. Esta ventana le da al sender una idea de cuánto espacio disponible hay en el buffer del receiver. Para que el sender pueda saber esa información, existe el campo receive window en los headers del segmento.

Supongamos que el host A le envía algo al host B. El host B va a tener un receive buffer de tamaño RcvBuffer, tal que $\text{LastByteRecv} - \text{LastByteRead} \leq \text{RcvBuffer}$. Receive window (rwnd) se setea con la cantidad de espacio libre, $\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRecv} - \text{LastByteRead})$. En cada segmento que envía B a A, le indica cuanto espacio tiene disponible. El host A "vigila" dos variables: LastByteSent y LastByteAcked. Manteniendo que $\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$, se asegura

que no

haya overflow en el buffer de B.

Podría suceder que se llene el buffer de B, se avise al host A, pero después no se tenga nada más que enviar desde B hasta A como para avisarle si se libera espacio. O sea, A quedaría bloqueado porque nunca se enteraría cuando hay espacio de nuevo. Para solucionar eso, se hace que A le siga mandando segmentos de un byte a B, así B los reconoce y va informando si se libera o no espacio.

[img]

[img]

[img]

Algunos escenarios en los que ocurre congestión:

1. Dos hosts que comparten un router con buffer infinito

[img]

[img]

[img]

El primer gráfico es el throughput por conexión (cantidad de bytes por segundo en el receiver) en función del rate de envío del sender. Con un sending rate entre 0 y $R/2$ (R es la capacidad del link), el throughput va a ser igual al rate de envío. Pero cuando el sending rate supera $R/2$, el throughput queda en $R/2$ porque está limitado por la capacidad del linker que está siendo utilizado por otra conexión además de esta.

En el segundo gráfico se ve que a medida que el rate de envío se acerca al valor de $R/2$, aumenta el delay porque aumenta la cantidad de paquetes encolados (este es un costo de la congestión).

2. Dos hosts y un router con buffer finito

Que el buffer sea finito significa que cuando se llene, los paquetes nuevos se van a perder. Pero también implica una conexión confiable, o sea que si se pierde un paquete, se va a retransmitir.

[img]

[img]

En el gráfico de la izquierda se asume que el sender retransmite paquetes solo cuando sabe que fueron perdidos. Un costo de una red congestionada es que el sender debe realizar retransmisiones para compensar por los paquetes perdidos por buffer overflow. En el gráfico de la derecha se asume que el sender puede cortar el timeout prematuramente y retransmitir paquetes que están demorados pero no perdidos. Ese es otro costo de la red congestionada, retransmisiones innecesarias que usan el bandwidth del link para reenviar copias innecesarias de paquetes.

3. 4 hosts, routers con buffers finitos y multihop paths

[img]

Acá se ve otro costo de soltar un paquete debido a la congestión: cuando un paquete se suelta en medio de un path, la capacidad de transmisión que fue usada en todos los links para llevar ese paquete a donde ahora fue arrojado, fue desperdiciada.

El approach que toma TCP para controlar la congestión es básicamente reducir o aumentar el rate de envío según qué tan congestionada esté la conexión.

TCP lleva registro de una variable que denominamos congestion window (cwnd). Esta ventana es la que limita el rate al que se puede enviar información desde el sender.

Debe suceder que

$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\} .$

Cómo percibe TCP que la red está congestionada? Lo que pasa cuando se congestiona es que se pierden paquetes en el camino, entonces ante un timeout o tres arribos de ACK por un mismo segmento, TCP va a percibir que hay congestión. A su vez, si no hay congestión lo va a percibir al recibir todos los ACK, y ante esto puede incrementar el congestion window. Como TCP utiliza los ACK para trigger el aumento o decremento de la ventana, se dice que es self-clocking.

Algunos guidelines:

A lost segment implies congestion, and hence, the TCP sender's rate should be decreased

when a segment is lost.

An acknowledged segment indicates that the network is delivering the sender's segments to

the receiver, and hence, the sender's rate can be increased when an ACK arrives for a previously unacknowledged segment.

Bandwidth probing. The TCP sender's behavior is perhaps analogous to the child

who requests (and gets) more and more goodies until finally he/she is finally told “No!”, backs off a bit, but then begins making requests again shortly afterwards.